Runtime Management

KC Sivaramakrishnan

IIT Madras

Monsoon 2026

The compiler: covered and coming next

Covered: lexical analysis, parsing, semantic analysis, and IR generation.

Remaining lectures, in teaching order:

- **08: Runtime management** — storage, activation records, calls, and non-local access.
- **09: Liveness and register allocation** — dataflow analysis, interference, colouring, and spilling.
- **10: Basic blocks and CFGs** — code generation, next-use information, dominators, and loops.
- **11: Local optimisation** — DAGs, common subexpressions, dead code, and peephole optimisation.
- **12: Global optimisation** — dataflow analysis, constant propagation, alias analysis, and loop optimisation.

These topics span the middle and back ends: IR analysis and optimisation, and target-code generation. Liveness supports register allocation.

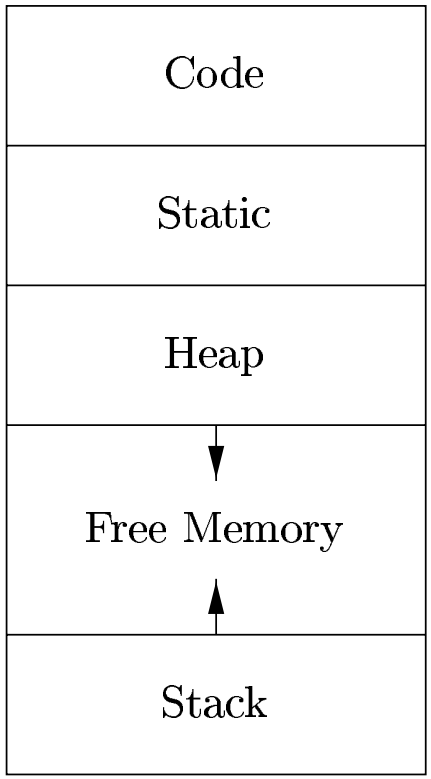
Management of run-time resources

- The compiler must generate code not only for computations, but also for managing the storage those computations use.
- Where are variables stored, and how is their storage allocated?
  - The code is generated at compile-time, but executes at run-time to perform the actual management.
  - Must take into account HLL concepts such as scopes, bindings, data types, procedure abstractions, etc.

- When a program is invoked:
  - The OS allocates space for the program
  - The code is loaded into part of the space
  - The OS jumps to the entry point (i.e. “main”)
- From this point onwards, the responsibility of managing the space falls to the program.

Run-time storage organization

Typical memory layout



The classical scheme

- allows both stack and heap maximal freedom
- code and static data may be separate or intermingled

Run-time storage organization

Code space

- fixed size
- statically allocated

Static space

- fixed-sized data may be statically allocated, e.g. global variables.
- Advantage is that addresses of such variables can be directly compiled into the machine code.
- variable-sized data must be dynamically allocated.

Heap and Stack

- Heap stores dynamically allocated objects
- Stack stores data structures called ‘activation records’ for maintaining procedure abstraction

The procedure abstraction

Separate compilation:

- allows us to build large programs
- keeps compile times reasonable
- requires independent procedures

The linkage convention:

- a social contract
- machine dependent
- division of responsibility

The linkage convention ensures that procedures inherit a valid run-time environment and that they restore one for their parents.

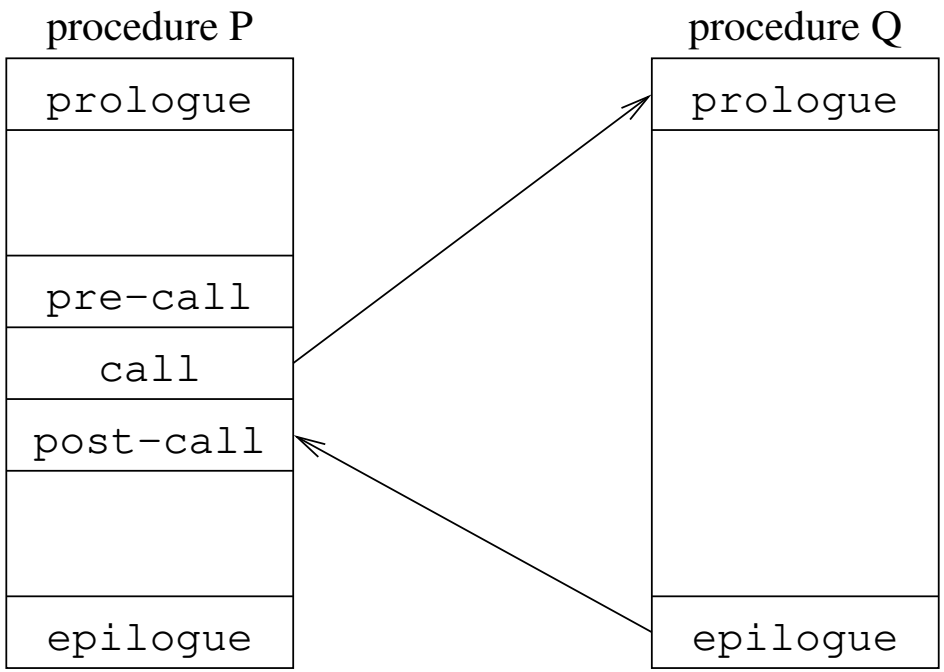
Linkages execute at run time

Code to make the linkage is generated at compile time

The procedure abstraction

The essentials:

- on entry, establish `p`'s environment
- at a call, preserve `p`'s environment
- on exit, tear down `p`'s environment
- in between, addressability and proper lifetimes



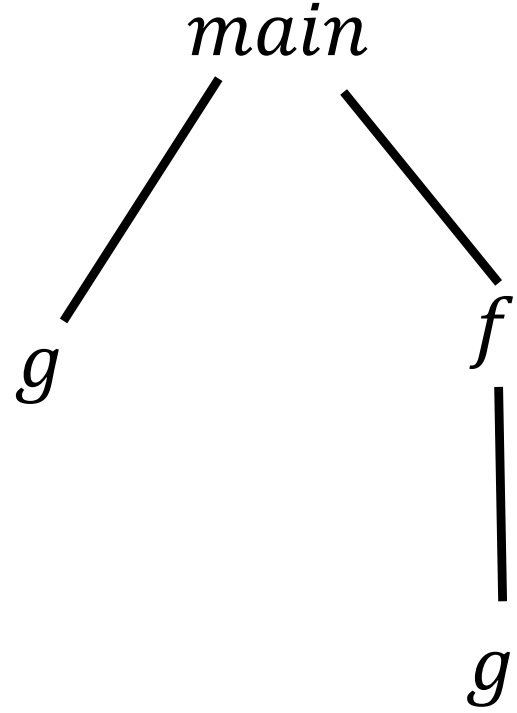
Each system has a standard linkage

Activations

- An invocation of procedure p is an activation of p.
- The lifetime of an activation of p consists of all the steps required to execute the activation.
  - Including all the steps in any procedure that p calls.
- Lifetimes of procedure activations are propertly nested.
  - Activation lifetimes can be depicted as a tree; called the activation tree.

Activation Tree: Example

```
class main{
public int g() { ... }
public int f() {
g();}
public int main() {
g(); f();
}
```



- Each node corresponds to an activation.
- Children of a node p correspond to activations of procedures called by the activation of p, in the order from left to right.

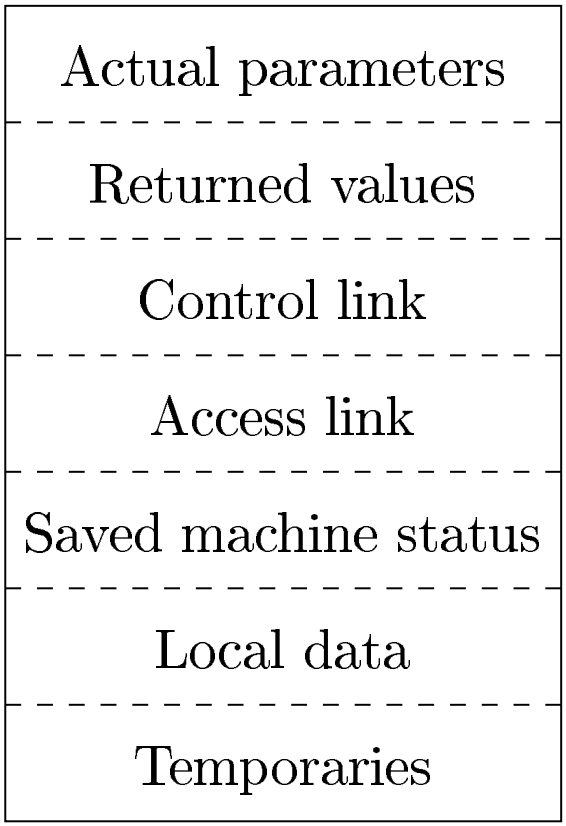
Activation Tree: Properties

- Activation tree is a dynamic structure; it depends on run-time behaviour.
  - The same program may have different activation trees in different executions.
- Several useful relationships exist between activation trees and runtime behaviour:
  - Sequence of procedure calls corresponds to pre-order traversal of the activation tree.
  - Sequence of returns corresponds to post-order traversal of the activation tree.
  - If the execution is currently in a particular activation p, then all activations that are currently live correspond to p and all its ancestors.
- We can use a stack to track live activations.

Activation Records

- The information needed to manage one procedure activation is called an activation record or frame.
- If a function f calls g, then g's activation record contains a mix of information about f and g.
  - f's execution is 'suspended' until g completes, after which, f must resume. Hence, g's AR contains information needed to resume execution of f.
  - g's AR will also contain space for return value, parameters and local variables of g.

A typical Activation Record



Hold the arguments supplied by the caller.

Hold the results returned to the caller.

Points to the caller's activation record.

Points to the AR of the lexical parent.

Saves the return address and register state.

Holds this invocation's local variables.

Hold intermediate results not kept in registers.

Note that this is one of many possible AR designs.
The compiler must fix the layout of the AR and generate code that correctly accesses locations in the AR.

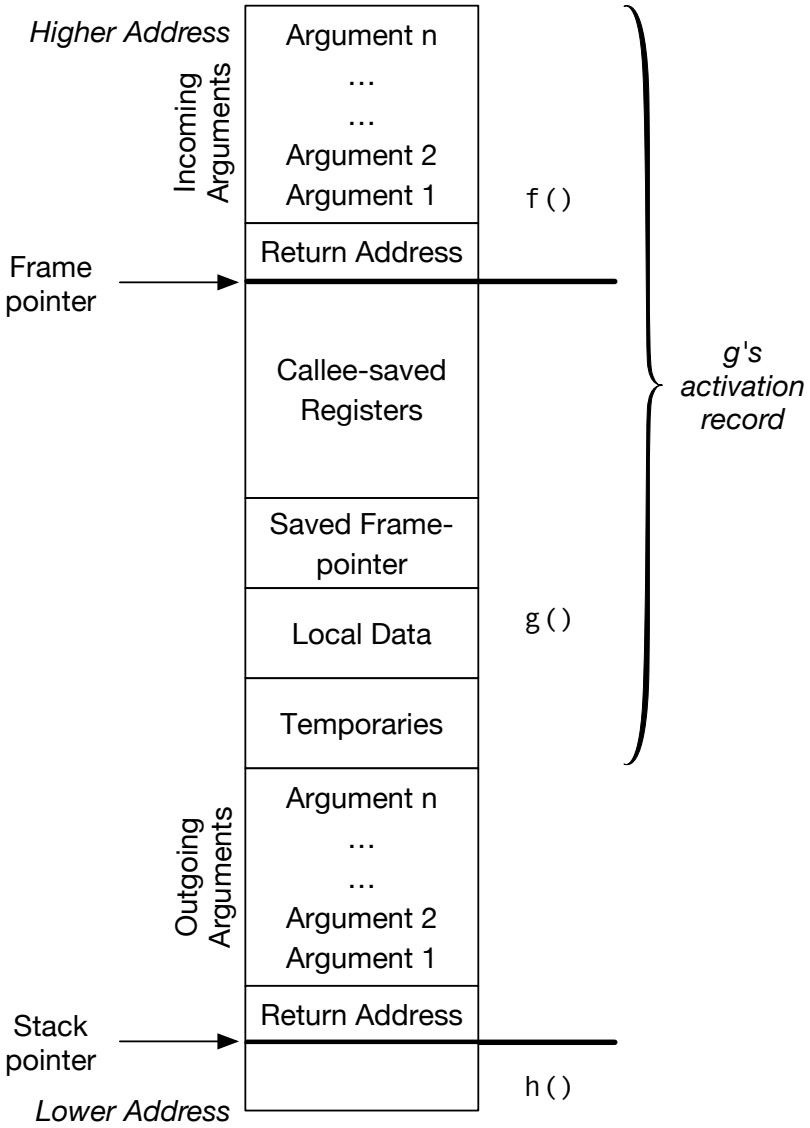
Procedure linkages

The linkage divides responsibility between caller and callee

	Caller	Callee
Call	pre-call	prologue
	1. allocate basic frame 2. evaluate & store params. 3. store return address 4. jump to child	1. save registers, state 2. store FP (dynamic link) 3. set new FP 4. Allocate space for local data
Return	post-call	epilogue
	1. copy return value 2. deallocate basic frame 3. restore parameters (if copy out)	1. store return value 2. restore state 3. restore parent's FP 4. jump to return address

At compile time, generate the code to do this.
At run time, that code manipulates the frame & data areas.

Activation records on the control stack



- Arguments are placed at the start of the AR; easy for caller to set them up.
- Temporaries are allocated at the end of the AR; current function can use as much space as necessary.
- The frame-pointer (FP) contains the base address of current frame
  - Arguments at constant positive offsets, local variables at constant negative offsets, relative to FP.

Discussion

- The set of steps performed while calling a procedure is called calling sequence.
  - Set of steps performed while returning is called return sequence.
- Note that there is noting magic about the division of responsibility between caller and callee.
  - One can choose a different division, as long as all the steps in calling and return sequence are carried out.
  - Choose an organization which optimizes some (or all parameters): Execution time, code length, ease of code generation, etc.
- To optimize code length, it is better to delegate as much responsibility to the callee as possible.
  - If a procedure is called n times, then the portion assigned to the caller would be generated n times.

Run-time storage organization

Where do local variables go?
When can we allocate them on a stack?
Key issue is lifetime of local names

Downward exposure:

- called procedures may reference my variables
- dynamic scoping
- lexical scoping

Upward exposure:

- can I return a reference to my variables?
- functions that return functions
- continuation-passing style

With only downward exposure, the compiler can allocate the frames on the run-time call stack

Storage classes

Each variable must be assigned a storage class

Statically allocated variables:

- include global variables and function-local `static` variables
- storage lasts for the entire program execution
- fixed-size storage layout determined before execution; addresses resolved by the compiler, linker, or loader
- shared data must have a consistent size, alignment, and layout across modules

Scope and linkage are separate from storage duration:

- scope determines where a name can be used
- linkage determines whether declarations refer to the same entity across scopes or modules

Storage classes (cont.)

Procedure local variables
Put them on the stack

- if sizes are fixed
- if lifetimes are limited
- if values are not preserved

Dynamically allocated variables
Must be treated differently

- call-by-reference, pointers, lead to non-local lifetimes
- (usually) an explicit allocation on the heap
- explicit or implicit deallocation

Access to non-local data

How does the code find non-local data at run-time?

Real globals

- visible everywhere
- naming convention gives an address
- initialization requires cooperation

Lexical nesting

- Allows nested procedure declarations
- A procedure can access variables of other procedures whose declaration surround its own declaration
- view variables as (nesting-depth,offset) pairs (compile-time)
- chain of non-local access links
- more expensive to find (at run-time)

Lexical Nesting: Example in OCaml

```
let sort inputFile outputFile =
  let a = Array.make 11 0 in
  let readArray inputFile =
    ... (* fills a from inputFile *)
  in
  let exchange i j =
    let tmp = a.(i) in
    a.(i) <- a.(j); a.(j) <- tmp
  in
  let rec quicksort m n =
    let v = a.(m) in
    let partition y z =
      ... (* uses a, v, and exchange *)
    in
    ... (* calls partition and quicksort *)
  in
  readArray inputFile;
  quicksort 1 9; (* matches the following diagrams *)
  ... (* writes a to outputFile *)
```

OCaml skeleton: . . . denotes omitted code.

Access to non-local data

Lexical nesting depth is fixed by the source code. In our example (numbering from 1):

Depth	Procedures
1	`sort`
2	`readArray`, `exchange`, `quicksort`
3	`partition`

Recursive calls to `quicksort` all have lexical depth 2.

Access to non-local data: two tasks

For now, functions are first-order: they may be nested and recursive, but calls name the function directly.

Compile-time mapping and run-time access:

- Resolve each variable reference at compile time. Search symbol tables from the innermost scope outward to find its declaration.
- Map the reference to (frame nesting depth, offset): the declaring function's depth and the variable's slot in its activation record. Inside `partition`: $v \mapsto (2, o_v)$ and $a \mapsto (1, o_a)$.
- Find the corresponding activation record at run time. Two classic approaches: access links (static links) and displays.

We introduce function values and closures after static links and displays.

Access to non-local data

To find the value specified by (l, o)

- current procedure has lexical nesting depth k (known at compile time)
- $k = l \Rightarrow$ local value
- $k > l \Rightarrow$ find l's activation record
- $k < l$ cannot occur

Maintaining access links (static links)

The callee's access link points to its lexical parent's frame. Here k is the caller's lexical nesting depth.

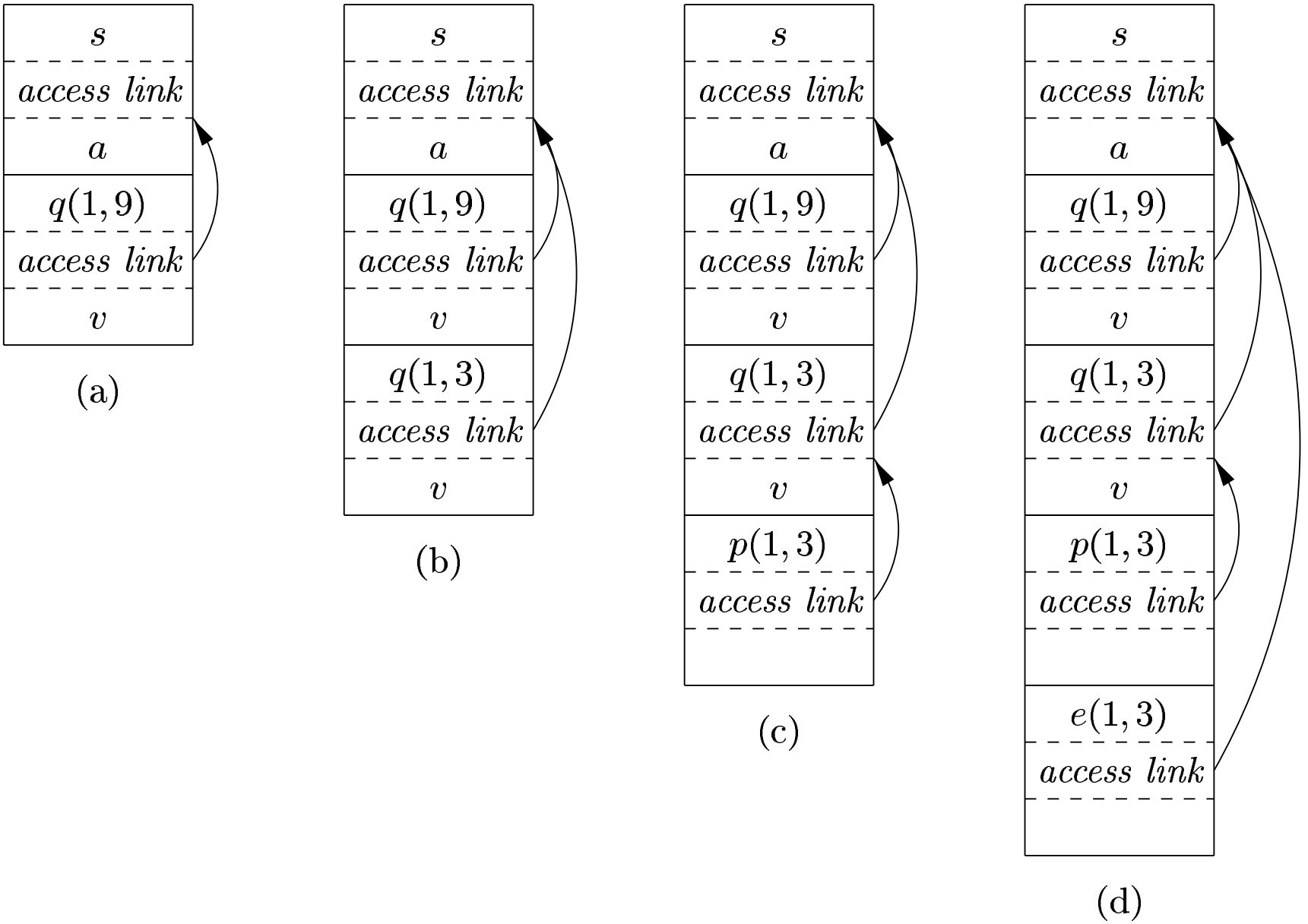
Calling a procedure at lexical nesting depth $k+1$:

- Pass the current frame pointer as the callee's access link.
- Example: `quicksort` (depth 2) calls `partition` (depth 3). The link points to this `quicksort` frame.

Calling a procedure at lexical nesting depth $l \leq k$:

- Follow access links to depth $l-1$; pass that frame pointer.
- Example: `quicksort` calls itself ($k = l = 2$). The new frame's link points to `sort` (depth 1).
- Example: `partition` calls `exchange` ($k = 3$, $l = 2$). Follow two links to `sort` (depth 1).

Using Access Links: Example



The display

To improve run-time access costs, use a display:

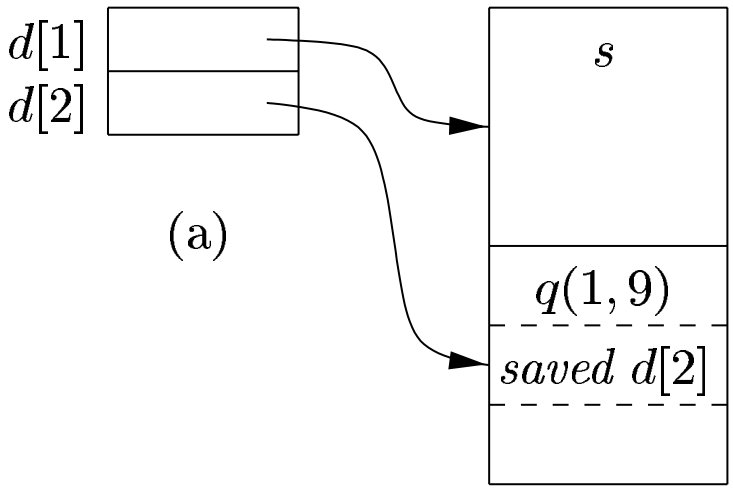
- table of access links for all nesting depths
- For each nesting-depth, the access link to the most recent activation record at the depth is stored.
- On entry, save the old slot in the new activation record and install its frame pointer; on return, restore the saved slot.
- Where to store the display? Can be stored separately or as part of the activation record.

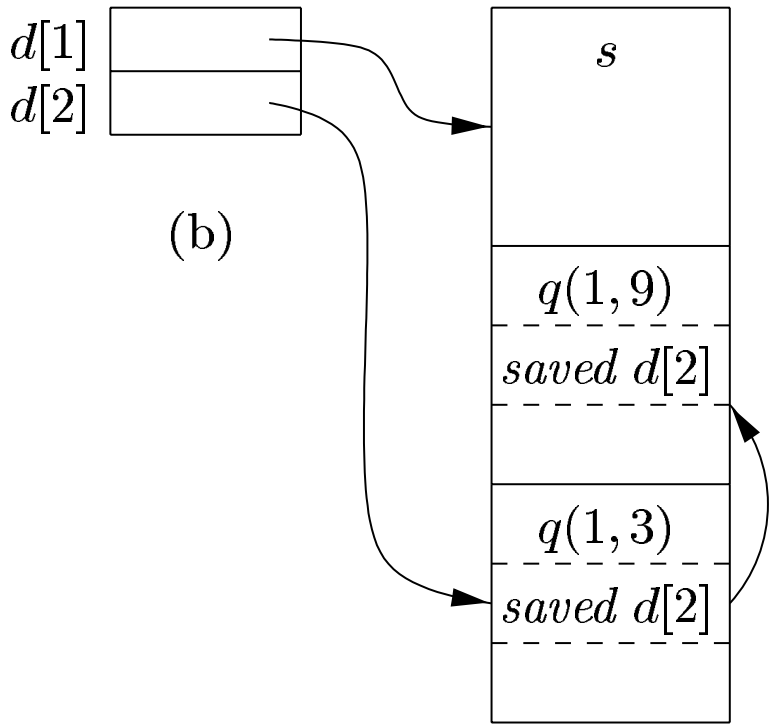
Access with the display
assume a value described by (l, o)

- find slot as `display`[l]
- add offset to pointer from slot (`display`[l][o])

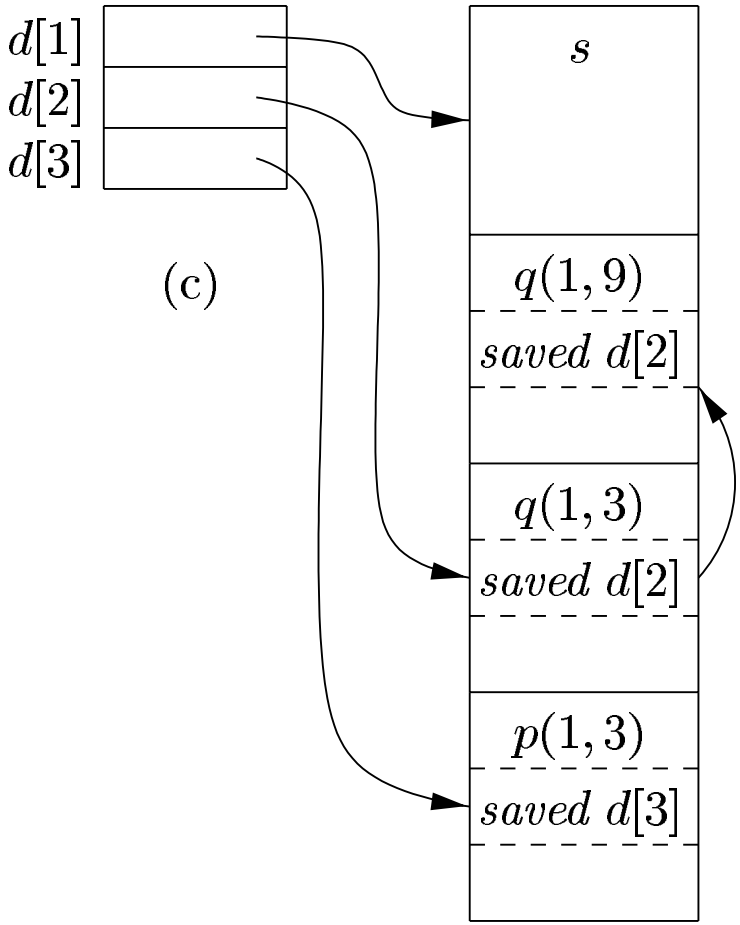
"Setting up the basic frame" now includes display manipulation

Display: Example

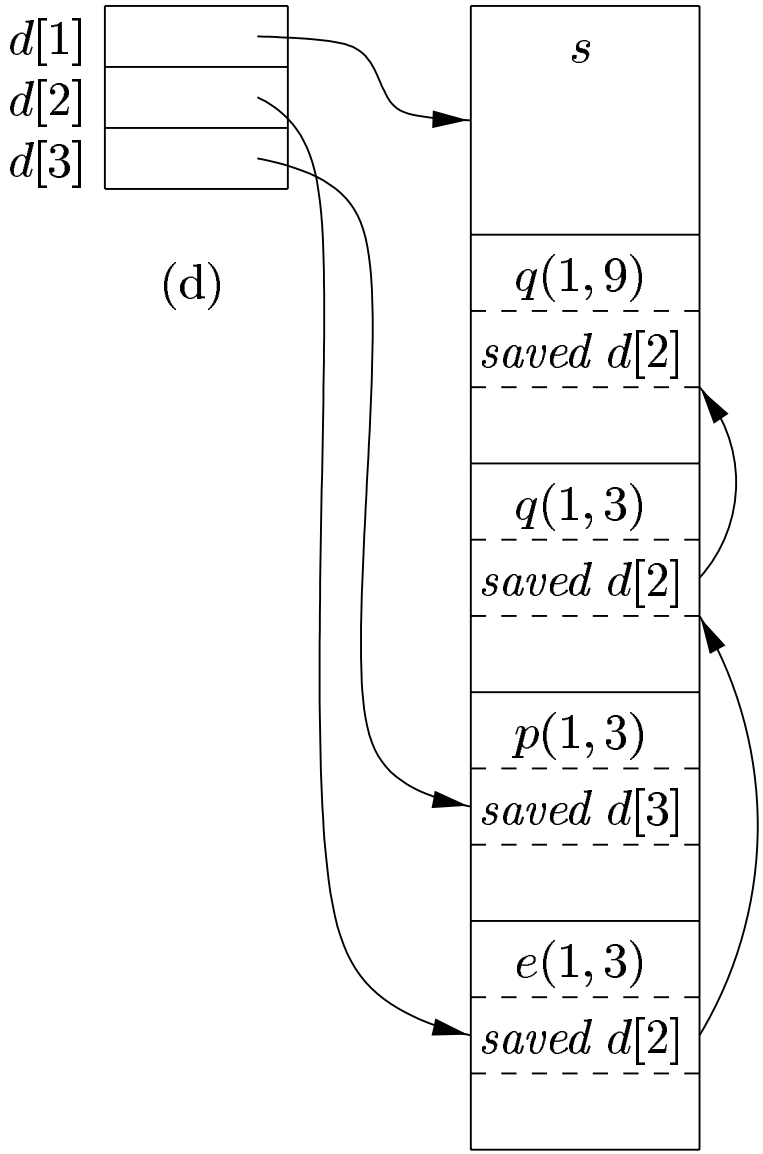




Display: Example



(c)



(d)

When the inner function outlives the outer call

```
let make_adder x =
  fun y -> x + y

let add10 = make_adder 10
let result = add10 5              (* 15 *)
```

- `make_adder` has returned when `add10` is called.
- Its stack frame is gone, but `x` must remain available.
- A link to that expired frame cannot provide `x`.

The function value must retain its captured environment.

Closure conversion: code plus environment

A closure pairs function code with its captured environment. Closure conversion makes that environment explicit.

```
adder_code(env, y):
    return env.x + y

make_adder(x):
    env = { x = x }
    return { code = adder_code, env = env }

add10 = make_adder(10)
result = add10.code(add10.env, 5)
```

- Access `x` at a fixed offset in `env`.
- Keep `env` alive for as long as the closure needs it.

Schematic pseudocode; physical layouts and calling conventions vary.

The quicksort example with closure environments

`partition` uses `a`, `v`, and `exchange`. One possible translation is:

```
(* inside each quicksort invocation *)
env = { a = a, v = v, exchange = exchange }
partition = { code = partition_code, env = env }

partition_code(env, y, z):
    ... env.a ... env.v ... env.exchange ...
```

- Each invocation captures its own pivot `v`.
- All environments refer to the same array `a`.
- Access captured fields directly, without a frame-chain walk.

Capturing `a` shares the array; it does not copy its contents.

Which representation does a compiler use?

- **Static chains remain in use.** GCC supports them for nested functions.
- **Closures carry captured environments.** OCaml uses closure conversion; Rust closures store captured values or references in fields.
- **Optimisation may remove the environment.** Inline the function, or pass captured values as extra arguments.

Captured variables are accessed at fixed offsets in the closure environment. In OCaml, the garbage collector keeps that environment alive while it is reachable.

Parameter passing

What about parameters?

Call-by-value

- store values, not addresses
- never restore on return
- arrays, structures, strings are a problem

Call-by-reference

- pass address
- access to formal is indirect reference to actual

Parameter passing

What about variable length argument lists?

1. if caller knows that callee expects a variable number
   1. caller can pass number as 0th parameter
   2. callee can find the number directly
2. if caller doesn't know anything about it
   1. callee must be able to determine number
   2. first parameter must be closest to `FP`

Consider `printf` :

- number of parameters determined by the format string
- it assumes the numbers match

Heap Management

- Data that outlives the procedure that creates it cannot be kept in the Activation record.
  - E.g. Objects created using `new` in Java, `malloc` in C/C++.
- Dynamically allocated data is stored on the heap.
- Heap Management techniques typically deal with allocation and deallocation problems.
  - Finding a contiguous segment of required size, avoiding fragmentation, etc.
  - Automatic/manual reclamation of space; Garbage Collection